01

Arquitetura Além do MVC

Laravel na prática

Do Controller sobrecarregado a casos de uso claros, testáveis e organizados.

Roteiro da aula

- Problema comum em projetos Laravel
- O limite do MVC básico
- Domain Driven Design Lite
- Organização por domínio
- Action Classes
- Controller limpo
- Data Transfer Objects
- View Models
- Monorepo, multirepo e critérios

Problema comum em projetos Laravel

Pontos

- Controllers acumulam validação, regra de negócio e chamadas externas.
- Models viram classes grandes demais.
- Views recebem dados crus e passam a decidir detalhes de exibição.
- Mudanças pequenas exigem editar muitos lugares.

O limite do MVC básico

Pontos

- MVC separa entrada, dados e apresentação.
- MVC não define sozinho onde ficam regras de negócio complexas.
- Laravel facilita com convenções, mas convenção não substitui arquitetura.

Exemplo

Exemplo de código

```
public function store(Request $request)
{
    $data = $request->validate([...]);

    if (Appointment::where(...)->exists()) {
        return back()->withErrors([...]);
    }

    $appointment = Appointment::create($data);
    return redirect()->route('appointments.show',
$appointment);
}
```

Domain Driven Design Lite

Definição

DDD Lite é uma forma pragmática de organizar código por áreas do negócio, sem aplicar toda a complexidade do DDD clássico.

Pontos

- Trocar pastas genéricas por módulos de domínio.
- Aproximar regras, dados, ações e apresentação da mesma área.
- Facilitar leitura por feature.

Estrutura exemplo

Exemplo de código

```
app/
  Domains/
    Appointments/
      Actions/
      Data/
      Models/
      ViewModels/
  Http/
    Controllers/
```

Organização por domínio

Pontos

- O desenvolvedor encontra tudo de uma feature mais rápido.
- O código revela melhor o negócio.
- A equipe reduz dependências acidentais entre partes do sistema.

Exemplo de código

```
ANTES
app/
  Http/Controllers/
  Models/

DEPOIS
app/
  Domains/Appointments/
  Http/Controllers/
```

Action Classes

Definição
Uma Action Class executa uma ação de negócio bem definida.

Exemplo

Pontos

- Controller recebe a requisição.
- Action executa o caso de uso.
- Testes ficam mais diretos.

Exemplo de código

```
final class CreateAppointmentAction
{
    public function execute(
        CreateAppointmentData $data
    ): Appointment {
        return DB::transaction(function () use ($data) {
            if (Appointment::where(...)->exists()) {
                throw new DomainException('Horário
indisponível.');
            }

            return Appointment::create([...]);
        });
    }
}
```

Controller limpo

Exemplo

Pontos

- O Controller coordena entrada e resposta.
- A Action concentra a regra da ação.
- O DTO transporta dados validados entre camadas.

Exemplo de código

```
public function store(
    Request $request,
    CreateAppointmentAction $action
) {
    $data = CreateAppointmentData::from(
        $request->validate([...])
    );

    $appointment = $action->execute($data);

    return redirect()->route(
        'appointments.show', $appointment
    );
}
```

Data Transfer Objects

Definição
DTO é um objeto usado para transportar dados com formato previsível.

Biblioteca
spatie/laravel-data

Exemplo

Pontos
- Evita arrays soltos circulando pela aplicação.
- Ajuda a documentar quais dados uma ação precisa.
- Reduz erros com chaves digitadas incorretamente.

Exemplo de código

```
use Spatie\LaravelData\Data;

final class CreateAppointmentData extends Data
{
    public function __construct(
        public readonly int $clinic_id,
        public readonly int $patient_id,
        public readonly int $professional_id,
        public readonly string $starts_at,
    ) {}
}
```

View Models

Definição

View Model prepara os dados que a tela precisa exibir.

Exemplo

Pontos

- Evite colocar formatação pesada na Blade.
- Reduz consultas e cálculos dentro da View.
- Ajuda a reutilizar preparação de dados.

Exemplo de código

```
final readonly class AppointmentShowViewModel
{
    public function __construct(
        private Appointment $appointment
    ) {}

    public function toArray(): array
    {
        return [
            'patient' => $this->appointment->patient->name,
            'status' => 'Agendado',
            'starts_at' => $this->appointment->starts_at
                ->format('d/m/Y H:i'),
        ];
    }
}
```

O mesmo produto em dois modelos

MONOREPO
sistema-agenda/
├── apps/api-laravel/
├── apps/painel-web/
├── apps/portal-paciente/
└── packages/contratos/
Uma mudança pode atualizar API, painel e testes no mesmo commit.

MULTIREPO
agenda-api
agenda-painel
agenda-portal-paciente
agenda-contratos
Cada aplicação possui seu próprio pipeline, versão e permissões.

Critérios de decisão

MONOREPO
Mudanças entre aplicações: mais simples de coordenar
CI/CD centralizado
Permissões menos granulares
Melhor cenário: uma equipe e aplicações que mudam juntas

MULTIREPO
Mudanças entre aplicações: exigem sincronização
CI/CD independente por aplicação
Permissões mais granulares
Melhor cenário: equipes e serviços independentes

Pergunta para a turma: a mesma mudança afeta API, painel e portal?

Projeto do dia

Produto usado
Sistema de Agenda, um SaaS de agendamento e atendimento para clínicas.

Missão
Refatorar um fluxo de criação de agendamentos que está concentrado no Controller.

Entregas

- Criar domínio Appointments.
- Criar CreateAppointmentAction.
- Criar CreateAppointmentData.
- Criar AppointmentShowViewModel.
- Comparar a estrutura MVC básica com a estrutura organizada por domínio.

Execução: composer setup (uma vez) → php artisan serve

Checklist final

Ao final, deve conseguir:

- Explicar por que MVC básico pode ficar insuficiente em projetos maiores.
- Organizar uma feature Laravel por domínio.
- Extrair regra de negócio para uma Action Class.
- Criar um DTO com spatie/laravel-data.
- Usar View Model para preparar dados de tela.
- Defender monorepo ou multirepo usando critérios técnicos.